# 🤖 MACHINE LEARNING

## From A to Z

A comprehensive cheatsheet covering all essential concepts, algorithms, techniques, and best practices in Machine Learning - from fundamentals to advanced topics.

📊 Supervised Learning | 🎯 Unsupervised Learning
🔄 Deep Learning | 📈 Model Evaluation
⚙️ Optimization | 🛠️ Feature Engineering

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

# What is Machine Learning?

> Machine Learning is a subset of AI that enables systems to learn and improve from experience without being explicitly programmed. It focuses on developing algorithms that can access data and use it to learn for themselves.

# Types of Machine Learning

### 🎓 Supervised Learning

Learn from labeled data

- Classification
- Regression

### 🎯 Unsupervised Learning

Find patterns in unlabeled data

- Clustering
- Dimensionality Reduction

### 🎮 Reinforcement Learning

Learn through rewards

- Agent-Environment
- Policy Learning

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

# The ML Workflow

1. **Problem Definition:** Define the business problem and success metrics
2. **Data Collection:** Gather relevant data from various sources
3. **Data Preprocessing:** Clean, transform, and prepare data
4. **Feature Engineering:** Create meaningful features
5. **Model Selection:** Choose appropriate algorithms
6. **Training:** Train model on training data
7. **Evaluation:** Assess model performance
8. **Hyperparameter Tuning:** Optimize model parameters
9. **Deployment:** Deploy model to production
10. **Monitoring:** Track performance and retrain as needed

# Key Terminology

| Term | Definition |
|---|---|
| **Features (X)** | Input variables used for prediction |
| **Target (y)** | Output variable to predict |
| **Model** | Mathematical function mapping features to target |
| **Training** | Process of learning from data |
| **Inference** | Making predictions on new data |

# Data Cleaning

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

## 📊 Handling Missing Values

**Strategies:**

- **Deletion:** Remove rows/columns with missing values
- **Imputation:** Fill with mean, median, mode, or predicted values
- **Forward/Backward Fill:** Use previous/next values (time series)
- **Interpolation:** Estimate values between known data points

```
# Python Examples df.dropna() # Remove missing df.fillna(df.mean()) # Mean imputation df.interpolate() # Interpolate values
```

## 🎯 Handling Outliers

**Detection Methods:**

- **Z-Score:** |Z| > 3 indicates outlier
- **IQR Method:** Values outside [Q1-1.5×IQR, Q3+1.5×IQR]
- **Isolation Forest:** ML-based detection

**Handling:** Remove, cap (winsorize), or transform

# Feature Scaling

### 📏 Normalization (Min-Max)

```
X' = (X - X_min) / (X_max - X_min)
```

Scales to [0, 1] range
**Use when:** Need bounded values, neural networks

### 📊 Standardization (Z-score)

```
X' = (X - μ) / σ
```

Mean=0, Std=1

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

**Use when:** Features have different scales, algorithms assume normal distribution

💡 **Tip:** Always fit scaler on training data only, then transform both train and test!

## Encoding Categorical Variables

| Method | When to Use | Example |
|---|---|---|
| **Label Encoding** | Ordinal data (order matters) | Low=0, Medium=1, High=2 |
| **One-Hot Encoding** | Nominal data (no order) | Color: Red=[1,0,0], Blue=[0,1,0] |
| **Target Encoding** | High cardinality categories | Replace with target mean |
| **Binary Encoding** | Many categories, less sparse | Convert to binary digits |

## Train-Test Split

**Purpose:** Evaluate model on unseen data to prevent overfitting

```
from sklearn.model_selection import train_test_split X_train, X_test, y_train, y_test = train_test_split( X, y, test_size=0.2, random_state=42, stratify=y )
```

⚠️ **Common Ratios:** 80-20, 70-30, 60-20-20 (train-val-test)

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

# Linear Regression

$$y = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \ldots + \beta_n x_n + \varepsilon \quad \text{Cost Function (MSE): } J(\beta) = (1/n)\ \Sigma(y_i - \hat{y}_i)^2$$

**Assumptions:**

- Linearity
- Independence
- Homoscedasticity
- Normality of errors
- No multicollinearity

```
from sklearn.linear_model import LinearRegression model = LinearRegression() model.fit(X_train, y_train) y_pred = model.predict(X_test)
```

# Regularized Regression

## 🎯 Ridge (L2)

$$J(\beta) = MSE + \alpha\ \Sigma\beta_i^2$$

**Effect:** Shrinks coefficients toward zero
**Use when:** Multicollinearity present

## 🎯 Lasso (L1)

$$J(\beta) = MSE + \alpha\ \Sigma|\beta_i|$$

**Effect:** Can zero out coefficients (feature selection)

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

**Use when:** Want sparse models

### 🎯 Elastic Net (L1 + L2)

$$J(\beta) = MSE + \alpha_1 \Sigma|\beta_i| + \alpha_2 \Sigma \beta_i^2$$

**Combines:** Benefits of both Ridge and Lasso

## Other Regression Algorithms

### 🌳 Decision Tree Regression

**Pros:** Non-linear, interpretable, no scaling needed
**Cons:** Prone to overfitting, unstable
**Key Parameters:** max_depth, min_samples_split, min_samples_leaf

### 🌲 Random Forest Regression

**Ensemble of decision trees**
**Pros:** Reduces overfitting, handles non-linearity well
**Key Parameters:** n_estimators, max_features, max_depth

### ⚡ Gradient Boosting (XGBoost, LightGBM)

**Sequential ensemble learning**
**Pros:** High performance, handles missing values
**Popular Libraries:** XGBoost, LightGBM, CatBoost

### 📊 Support Vector Regression (SVR)

**Uses epsilon-tube for predictions**
**Pros:** Effective in high dimensions, robust to outliers
**Key Parameters:** kernel (linear, rbf, poly), C, epsilon

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

# Regression Metrics

| Metric | Formula | Interpretation |
|---|---|---|
| **MAE** | $(1/n)\ \Sigma\lvert y_i - \hat{y}_i\rvert$ | Average absolute error |
| **MSE** | $(1/n)\ \Sigma(y_i - \hat{y}_i)^2$ | Penalizes large errors |
| **RMSE** | $\sqrt{MSE}$ | Same units as target |
| **$R^2$ Score** | 1 - (SS_res / SS_tot) | Variance explained (0-1) |

# Logistic Regression

$$\text{Sigmoid: } \sigma(z) = 1 / (1 + e^{-z}) \text{ where } z = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \ldots + \beta_n x_n \quad P(y=1|x) = \sigma(z)$$

**Binary Classification:** Predicts probability of class 1
**Decision Boundary:** P = 0.5 (adjustable threshold)
**Loss:** Log Loss (Cross-Entropy)

# Classification Algorithms

### 👥 K-Nearest Neighbors (KNN)

**Idea:** Classify based on k closest neighbors
**Pros:** Simple, no training phase
**Cons:** Slow prediction, sensitive to scale
**Key Param:** k (number of neighbors)

### 📊 Naive Bayes

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

**Idea:** Based on Bayes' theorem
**Pros:** Fast, works well with small data
**Cons:** Assumes feature independence
**Types:** Gaussian, Multinomial, Bernoulli

### 🎯 Support Vector Machine (SVM)

**Idea:** Find optimal hyperplane
**Pros:** Effective in high dimensions
**Cons:** Slow with large datasets
**Kernels:** Linear, RBF, Polynomial

### 🌳 Decision Trees

**Idea:** Tree-like decision rules
**Pros:** Interpretable, no scaling needed
**Cons:** Overfits easily
**Criteria:** Gini, Entropy

### 🌲 Random Forest

**Idea:** Ensemble of decision trees
**Pros:** High accuracy, reduces overfitting
**Cons:** Less interpretable, slower
**Method:** Bagging + Random features

### ⚡ Gradient Boosting

**Idea:** Sequential error correction
**Pros:** State-of-the-art performance
**Cons:** Prone to overfitting, tuning needed
**Libraries:** XGBoost, LightGBM, CatBoost

## Classification Metrics

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

## 📊 Confusion Matrix

| | Predicted Positive | Predicted Negative |
|---|---|---|
| **Actual Positive** | True Positive (TP) | False Negative (FN) |
| **Actual Negative** | False Positive (FP) | True Negative (TN) |

```
Accuracy = (TP + TN) / Total Precision = TP / (TP + FP) Recall (Sensitivity) = TP / (TP + FN) F1-Score = 2 × (Precision × Recall) / (Precision + Recall)
```

**When to use:**

- **Accuracy:** Balanced classes
- **Precision:** Cost of FP high
- **Recall:** Cost of FN high
- **F1:** Balance precision/recall
- **ROC-AUC:** Overall performance

# Clustering Algorithms

## 🎯 K-Means Clustering

**Algorithm:**

1. Initialize k cluster centers randomly
2. Assign each point to nearest center
3. Update centers as mean of assigned points
4. Repeat 2-3 until convergence

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

**Pros:** Fast, scalable, simple
**Cons:** Assumes spherical clusters, needs k specified
**Choosing k:** Elbow method, Silhouette score

```
from sklearn.cluster import KMeans kmeans = KMeans(n_clusters=3, random_state=42) labels = kmeans.fit_predict(X)
```

## 🌲 Hierarchical Clustering

**Agglomerative (Bottom-up):**

- Start: Each point is a cluster
- Merge closest clusters iteratively
- Result: Dendrogram

**Divisive (Top-down):**

- Start: All points in one cluster
- Split clusters recursively
- Less common approach

**Linkage Methods:** Single, Complete, Average, Ward

## 📍 DBSCAN (Density-Based)

**Key Concepts:**

- **Core Points:** ≥ min_samples within epsilon
- **Border Points:** Near core points
- **Noise Points:** Neither core nor border

**Pros:** Finds arbitrary shapes, detects outliers
**Cons:** Struggles with varying densities
**Parameters:** epsilon (radius), min_samples

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

## 🎲 Gaussian Mixture Models (GMM)

**Soft clustering:** Assigns probability of belonging to each cluster
**Uses:** Expectation-Maximization (EM) algorithm
**Advantage:** More flexible than K-Means (elliptical clusters)

# Dimensionality Reduction

## 📊 Principal Component Analysis (PCA)

**Purpose:** Find directions of maximum variance
**Steps:**

1. Standardize data
2. Compute covariance matrix
3. Find eigenvectors & eigenvalues
4. Select top k components
5. Transform data

**Use cases:** Visualization, noise reduction, feature extraction

```
from sklearn.decomposition import PCA pca = PCA(n_components=2) X_reduced = pca.fit_transform(X) print(f"Variance explained: {pca.explained_variance_ratio_}")
```

## 🗺️ t-SNE (t-Distributed Stochastic Neighbor Embedding)

**Purpose:** Non-linear dimensionality reduction for visualization
**Best for:** 2D/3D visualization of high-dimensional data
**Key Parameter:** perplexity (5-50 typical)
**Note:** Computationally expensive, non-deterministic

## 🎯 Linear Discriminant Analysis (LDA)

**Supervised dimensionality reduction**
Maximizes class separability (unlike PCA)

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

**Max components:** min(n_features, n_classes - 1)

## Clustering Evaluation Metrics

| Metric | Range | Description |
|---|---|---|
| **Silhouette Score** | [-1, 1] | Measures cluster cohesion & separation (higher is better) |
| **Davies-Bouldin Index** | [0, ∞] | Average similarity between clusters (lower is better) |
| **Calinski-Harabasz** | [0, ∞] | Ratio of between/within cluster dispersion (higher is better) |

## Neural Network Basics

**Components:**

- **Neurons:** Basic computational units
- **Layers:** Input, Hidden, Output
- **Weights & Biases:** Learnable parameters
- **Activation Functions:** Introduce non-linearity

**Neuron Output:** $a = f(\Sigma(w_i x_i) + b)$ where: - $w$ = weights - $x$ = inputs - $b$ = bias - $f$ = activation function

## Activation Functions

### 🔢 Sigmoid

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

```
σ(x) = 1/(1 + e⁻ˣ)
```

**Range:** (0, 1)
**Use:** Output layer (binary classification)
**Issue:** Vanishing gradients

## 📈 Tanh

```
tanh(x) = (eˣ - e⁻ˣ)/(eˣ + e⁻ˣ)
```

**Range:** (-1, 1)
**Use:** Hidden layers
**Better than sigmoid:** Zero-centered

## ⚡ ReLU (Rectified Linear Unit)

```
ReLU(x) = max(0, x)
```

**Range:** [0, ∞)
**Pros:** Fast, reduces vanishing gradient
**Cons:** Dead neurons (x < 0)

## 🔥 Leaky ReLU

```
f(x) = max(0.01x, x)
```

**Range:** (-∞, ∞)
**Advantage:** Solves dying ReLU problem
**Variants:** PReLU, ELU

## 🎯 Softmax

```
σ(xᵢ) = eˣⁱ / Σeˣʲ
```

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

**Range:** (0, 1), sum = 1
**Use:** Multi-class classification output
**Output:** Probability distribution

# Backpropagation & Optimization

## 🔄 Backpropagation

**Algorithm:**

1. Forward pass: Compute predictions
2. Calculate loss
3. Backward pass: Compute gradients (chain rule)
4. Update weights using gradients

## ⚙️ Optimization Algorithms

### 📉 Gradient Descent

$$\theta = \theta - \alpha \nabla J(\theta)$$

**Variants:**

- Batch GD: Use all data
- SGD: One sample at a time
- Mini-batch: Small batches

### 🚀 Adam (Adaptive Moment)

Combines momentum + RMSprop
**Most popular optimizer**
**Hyperparameters:**

- lr = 0.001 (default)

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

- $\beta_1 = 0.9$, $\beta_2 = 0.999$

### ⚡ SGD with Momentum

```
v = βv + α∇J(θ)  θ = θ − v
```

Accelerates convergence

### 📊 RMSprop

Adaptive learning rate
Works well with RNNs
Good for non-stationary objectives

## Loss Functions

| Task | Loss Function | Formula |
|---|---|---|
| Binary Classification | Binary Cross-Entropy | $-[y \log(\hat{y}) + (1-y) \log(1-\hat{y})]$ |
| Multi-class Classification | Categorical Cross-Entropy | $-\Sigma\, y_i \log(\hat{y}_i)$ |
| Regression | Mean Squared Error (MSE) | $(1/n)\, \Sigma(y_i - \hat{y}_i)^2$ |
| Regression | Mean Absolute Error (MAE) | $(1/n)\, \Sigma\lvert y_i - \hat{y}_i \rvert$ |

## Convolutional Neural Networks (CNNs)

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

**Purpose:** Process grid-like data (images, video)
**Key Idea:** Local connectivity + parameter sharing

## 🔍 CNN Layers

### 1. Convolutional Layer

- Applies filters/kernels
- Detects features (edges, textures)
- **Parameters:** filters, kernel_size, stride, padding
- **Output size:** [(W - K + 2P)/S] + 1

### 2. Pooling Layer

- Downsamples feature maps
- **Types:** Max Pooling, Average Pooling
- Reduces spatial dimensions
- Provides translation invariance

### 3. Fully Connected Layer

- Standard neural network layer
- Typically at the end
- Combines features for classification

### 4. Batch Normalization

- Normalizes layer inputs
- Speeds up training
- Reduces internal covariate shift

## 🏗️ Famous CNN Architectures

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

| Architecture | Year | Key Innovation |
|---|---|---|
| **LeNet-5** | 1998 | First successful CNN |
| **AlexNet** | 2012 | ReLU, Dropout, GPU training |
| **VGGNet** | 2014 | Very deep (16-19 layers), small filters (3×3) |
| **ResNet** | 2015 | Skip connections, 152+ layers |
| **Inception** | 2014 | Multiple filter sizes in parallel |

```
# Simple CNN in Keras/TensorFlow from tensorflow.keras import layers, models model = models.Sequential([ layers.Conv2D(32, (3,3), activation='relu', input_shape=(28,28,1)), layers.MaxPooling2D((2,2)), layers.Conv2D(64, (3,3), activation='relu'), layers.MaxPooling2D((2,2)), layers.Flatten(), layers.Dense(64, activation='relu'), layers.Dense(10, activation='softmax') ])
```

# Recurrent Neural Networks (RNNs)

**Purpose:** Process sequential data (text, time series, audio)
**Key Feature:** Memory of previous inputs through hidden state

**RNN Hidden State:** $h_t = \tanh(W_{xx}x_t + W_{hh}h_{t-1} + b)$

## 🔄 RNN Variants

### 📝 LSTM (Long Short-Term Memory)

**Problem Solved:** Vanishing gradient in long sequences
**Gates:**

- Forget gate: What to forget
- Input gate: What to update
- Output gate: What to output

**Use:** Long sequences, better gradient flow

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

### ⚡ GRU (Gated Recurrent Unit)

**Simpler than LSTM:** 2 gates instead of 3
**Gates:**

- Update gate
- Reset gate

**Advantage:** Faster training, fewer parameters

## 🔄 RNN Architectures

### One-to-One

Standard NN
Example: Image classification

### One-to-Many

One input → sequence output
Example: Image captioning

### Many-to-One

Sequence → single output
Example: Sentiment analysis

### Many-to-Many

Sequence → sequence
Example: Machine translation

# Cross-Validation Techniques

## 🔄 K-Fold Cross-Validation

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

**Process:**

1. Split data into k equal folds
2. For each fold: use it as validation, train on rest
3. Average performance across k iterations

**Common k values:** 5 or 10
**Advantage:** Every sample used for both training and validation

```
from sklearn.model_selection import cross_val_score scores = cross_val_score(model, X, y, cv=5, scoring='accuracy') print(f"Mean: {scores.mean():.3f}, Std: {scores.std():.3f}")
```

### 🎯 Stratified K-Fold

**Use:** Classification with imbalanced classes
**Ensures:** Each fold has same class distribution
**Benefit:** More reliable estimates

### 📊 Time Series Split

**Use:** Temporal data
**Preserves:** Chronological order
**Avoids:** Data leakage from future

## Bias-Variance Tradeoff

$$\text{Total Error} = \text{Bias}^2 + \text{Variance} + \text{Irreducible Error}$$

### 📉 High Bias (Underfitting)

**Symptoms:**

- High training error
- High validation error

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

- Model too simple

**Solutions:**

- Add more features
- Use more complex model
- Reduce regularization

### 📈 High Variance (Overfitting)

**Symptoms:**

- Low training error
- High validation error
- Model too complex

**Solutions:**

- More training data
- Regularization (L1, L2, Dropout)
- Simplify model
- Early stopping

## Regularization Techniques

### 🎯 L1 Regularization (Lasso)

```
Loss = MSE + λΣ|wi|
```

**Effect:** Sparse models (some weights = 0)
**Use:** Feature selection

### 🎯 L2 Regularization (Ridge)

```
Loss = MSE + λΣwi²
```

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

**Effect:** Weights shrink toward zero
**Use:** Reduce model complexity

### 💧 Dropout

**Method:** Randomly drop neurons during training
**Rate:** Typically 0.2-0.5
**Use:** Neural networks

### 🛑 Early Stopping

**Method:** Stop when validation error increases
**Monitor:** Validation loss
**Patience:** Wait n epochs before stopping

## Hyperparameter Tuning

### 🔍 Grid Search

**Method:** Try all combinations
**Pros:** Exhaustive, simple
**Cons:** Computationally expensive

### 🎲 Random Search

**Method:** Random parameter sampling
**Pros:** Faster, often better
**Cons:** Not exhaustive

### 🎯 Bayesian Optimization

**Method:** Model-based optimization
**Pros:** Efficient, smart search
**Libraries:** Optuna, Hyperopt

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

```
from sklearn.model_selection import GridSearchCV param_grid = {
'n_estimators': [100, 200, 300], 'max_depth': [10, 20, 30],
'min_samples_split': [2, 5, 10] } grid_search = GridSearchCV(
RandomForestClassifier(), param_grid, cv=5, scoring='accuracy' )
grid_search.fit(X_train, y_train) print(f"Best params:
{grid_search.best_params_}")
```

# Feature Creation Techniques

## 🔢 Numerical Features

**Mathematical Transformations:**

- Log transform: $\log(x + 1)$
- Square root: $\sqrt{x}$
- Power transform: $x^2$, $x^3$
- Box-Cox transform

**Binning/Discretization:**

- Equal-width binning
- Equal-frequency binning
- Custom bins based on domain knowledge

**Interaction Features:**

- Multiplication: $x_1 \times x_2$
- Division: $x_1 / x_2$
- Polynomial features

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

**Aggregation Features:**

- Sum, mean, median
- Standard deviation
- Min, max, range

## 📅 Date/Time Features

### Extract Components

- Year, Month, Day
- Hour, Minute, Second
- Day of week
- Quarter
- Week of year

### Cyclical Features

- Sin/Cos encoding
- Preserves cyclical nature
- Example: Hour 23 → 0

```
sin_hour = sin(2π × hour/24) cos_hour = cos(2π × hour/24)
```

### Temporal Features

- Time since event
- Is weekend/holiday
- Time of day categories
- Seasons

## 📝 Text Features

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

### Bag of Words (BoW)

Frequency-based representation
**CountVectorizer:** Word counts
**Limitation:** Ignores word order

### TF-IDF

```
TF-IDF = TF × IDF TF = (freq of term in doc) IDF = log(N / docs with term)
```

Weighs importance of words

### N-grams

**Unigrams:** Single words
**Bigrams:** Word pairs
**Trigrams:** Word triplets
Captures some context

### Word Embeddings

**Word2Vec, GloVe:** Dense vectors
**BERT, GPT:** Contextual embeddings
Captures semantic meaning

## Feature Selection

### 🎯 Filter Methods

**Before training**

- Correlation
- Chi-square test
- Information gain
- Variance threshold

**Pros:** Fast, model-agnostic

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

### 🔄 Wrapper Methods

**Use model performance**

- Forward selection
- Backward elimination
- Recursive feature elimination

**Pros:** Considers feature interactions
**Cons:** Computationally expensive

### 🌲 Embedded Methods

**During training**

- Lasso (L1)
- Tree-based feature importance
- ElasticNet

**Pros:** Balance of speed & accuracy

```
# Feature importance from Random Forest from sklearn.ensemble import
RandomForestClassifier rf = RandomForestClassifier(n_estimators=100)
rf.fit(X_train, y_train) feature_importance = pd.DataFrame({ 'feature':
X.columns, 'importance': rf.feature_importances_
}).sort_values('importance', ascending=False)
print(feature_importance.head(10))
```

## Handling Imbalanced Data

### 📊 Resampling Techniques

**Undersampling:**

- Random undersampling
- Tomek links
- NearMiss

**Oversampling:**

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

- Random oversampling
- SMOTE (Synthetic Minority)
- ADASYN

### ⚖️ Algorithm-Level

**Class weights:**

- Penalize minority class errors more
- class_weight='balanced'

**Ensemble methods:**

- BalancedRandomForest
- EasyEnsemble

**Threshold moving:**

- Adjust decision threshold

## What is Ensemble Learning?

**Idea:** Combine multiple models to improve performance
**Principle:** "Wisdom of the crowd" - diverse models make better predictions together
**Goal:** Reduce bias, variance, or both

## Bagging (Bootstrap Aggregating)

**Process:**

1. Create multiple bootstrap samples (random sampling with replacement)
2. Train a model on each sample
3. Aggregate predictions (voting/averaging)

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

**Effect:** Reduces variance (helps with overfitting)
**Best for:** High-variance models (e.g., decision trees)

### 🌲 Random Forest

**Bagging + Random Feature Selection**

- Each tree trained on bootstrap sample
- Each split considers random feature subset
- Final prediction: majority vote (classification) or average (regression)

```
from sklearn.ensemble import RandomForestClassifier rf = RandomForestClassifier( n_estimators=100, max_depth=10, min_samples_split=2, max_features='sqrt', random_state=42 ) rf.fit(X_train, y_train)
```

**Key Parameters:**

- **n_estimators:** Number of trees (100-500 typical)
- **max_features:** Features per split ('sqrt', 'log2', or number)
- **max_depth:** Tree depth (controls overfitting)
- **min_samples_split:** Min samples to split node

## Boosting

**Idea:** Sequentially train models, each correcting errors of previous ones
**Effect:** Reduces bias (improves weak learners)
**Process:** Weight misclassified examples more in next iteration

### ⚡ AdaBoost (Adaptive Boosting)

**Algorithm:**

1. Start with equal weights for all samples

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

2. Train weak learner
3. Increase weights of misclassified samples
4. Repeat for n iterations
5. Final prediction: weighted vote

**Best for:** Binary classification with simple base learners

## 🚀 Gradient Boosting

**Idea:** Fit new model to residuals (errors) of previous models

**Process:**

1. Train initial model
2. Calculate residuals (actual - predicted)
3. Train new model on residuals
4. Add to ensemble with learning rate
5. Repeat

$$F(x) = F_0(x) + \eta \cdot h_1(x) + \eta \cdot h_2(x) + \ldots + \eta \cdot h_m(x) \text{ where } \eta = \text{learning rate, } h_i = \text{weak learner}$$

## 🏆 XGBoost (Extreme Gradient Boosting)

**Improvements over GB:**

- Regularization (L1, L2)
- Parallel processing
- Tree pruning
- Built-in cross-validation
- Handling missing values

```
import xgboost as xgb model = xgb.XGBClassifier( n_estimators=100, learning_rate=0.1, max_depth=6, subsample=0.8, colsample_bytree=0.8,
```

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

```
gamma=0, reg_alpha=0, # L1 reg_lambda=1 # L2 ) model.fit(X_train, y_train)
```

## 💡 LightGBM & CatBoost

### LightGBM

**Key Features:**

- Leaf-wise tree growth (vs level-wise)
- Histogram-based algorithm
- Faster training on large datasets
- Lower memory usage

### CatBoost

**Key Features:**

- Handles categorical features natively
- Ordered boosting (reduces overfitting)
- Robust to hyperparameters
- Less tuning required

# Stacking

**Idea:** Use a meta-model to combine predictions of base models
**Process:**

1. Train multiple base models (Level 0)
2. Generate predictions from base models
3. Use predictions as features for meta-model (Level 1)
4. Train meta-model on these features

**Advantage:** Combines strengths of different algorithm types

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

```
from sklearn.ensemble import StackingClassifier from sklearn.linear_model
import LogisticRegression from sklearn.tree import DecisionTreeClassifier
from sklearn.svm import SVC estimators = [ ('rf',
RandomForestClassifier(n_estimators=100)), ('dt',
DecisionTreeClassifier(max_depth=5)), ('svm', SVC(probability=True)) ]
stacking_model = StackingClassifier( estimators=estimators,
final_estimator=LogisticRegression() ) stacking_model.fit(X_train, y_train)
```

# Voting

### Hard Voting

Majority vote wins
**Example:** 3 models predict [1, 1, 0] → Output: 1
**Use:** Classification

### Soft Voting

Average predicted probabilities
**Example:** [0.7, 0.6, 0.4] → Average: 0.57
**Use:** When models output probabilities

# Transfer Learning

**Idea:** Use knowledge from pre-trained models on new tasks
**When to use:** Limited data, computationally expensive training

### 🔧 Feature Extraction

**Method:**

- Use pre-trained model as feature extractor
- Freeze all layers

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

- Add new classifier on top
- Train only new layers

**Use when:** Very limited data

### 🎨 Fine-tuning

**Method:**

- Start with pre-trained weights
- Unfreeze some/all layers
- Train with small learning rate
- Adapt to new task

**Use when:** Moderate amount of data

```
# Transfer learning with Keras from tensorflow.keras.applications import VGG16 # Load pre-trained model base_model = VGG16(weights='imagenet', include_top=False, input_shape=(224, 224, 3)) # Freeze base layers base_model.trainable = False # Add custom layers model = models.Sequential([ base_model, layers.GlobalAveragePooling2D(), layers.Dense(256, activation='relu'), layers.Dropout(0.5), layers.Dense(num_classes, activation='softmax') ])
```

# Attention Mechanisms & Transformers

## 👁️ Attention Mechanism

**Problem:** RNNs struggle with long sequences (information bottleneck)
**Solution:** Allow model to "focus" on relevant parts of input
**Key Idea:** Weighted sum of all inputs based on relevance

```
Attention(Q, K, V) = softmax(QK^T / √d_k) × V where: Q = Query, K = Key, V = Value d_k = dimension of key
```

## 🤖 Transformer Architecture

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

**Revolutionary for NLP (2017)**
**Key Components:**

- **Self-Attention:** Relate different positions in sequence
- **Multi-Head Attention:** Multiple attention mechanisms in parallel
- **Positional Encoding:** Add position information
- **Feed-Forward Networks:** Process attended information

**Advantages:**

- Parallelizable (faster training)
- Handles long-range dependencies
- No sequential processing required

## 🌟 Famous Transformer Models

| Model | Type | Key Features |
|---|---|---|
| **BERT** | Encoder-only | Bidirectional, masked language modeling, classification |
| **GPT** | Decoder-only | Autoregressive, text generation, few-shot learning |
| **T5** | Encoder-Decoder | Text-to-text framework, versatile |
| **ViT** | Vision | Transformers for images, patches as tokens |

# Generative AI

### 🎨 Generative Adversarial Networks (GANs)

**Components:**

- **Generator:** Creates fake data
- **Discriminator:** Distinguishes real vs fake

**Training:** Adversarial game - generator tries to fool discriminator

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

**Applications:** Image generation, style transfer, data augmentation

### 🔀 Variational Autoencoders (VAEs)

**Components:**

- **Encoder:** Maps input to latent space
- **Decoder:** Reconstructs from latent

**Key:** Learn continuous latent representations
**Applications:** Generation, anomaly detection

### 🌊 Diffusion Models

**Process:**

- Forward: Gradually add noise
- Reverse: Learn to denoise

**State-of-the-art:** Image generation (DALL-E, Stable Diffusion)

### 🗣️ Large Language Models (LLMs)

**Examples:** GPT-4, Claude, Llama
**Capabilities:**

- Text generation
- Question answering
- Translation, summarization
- Code generation

## MLOps Overview

**MLOps:** Practices for deploying and maintaining ML models in production
**Goal:** Reliable, scalable, and reproducible ML systems

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

**Combines:** ML, DevOps, Data Engineering

# ML Pipeline Components

### 📊 Data Pipeline

- Data ingestion
- Data validation
- Data preprocessing
- Feature engineering
- Data versioning

### 🤖 Model Pipeline

- Model training
- Hyperparameter tuning
- Model evaluation
- Model versioning
- Model registry

### 🚀 Deployment Pipeline

- Model packaging
- Containerization
- A/B testing
- Monitoring
- Rollback mechanisms

# Model Deployment Strategies

### 🔵 Batch Prediction

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

**When:** Predictions not needed in real-time
**Process:**

- Run predictions on schedule
- Store results in database
- Serve pre-computed predictions

**Examples:** Daily recommendations, weekly reports

### ⚡ Online/Real-time Prediction

**When:** Instant predictions required
**Methods:**

- REST API
- gRPC
- Serverless functions

**Examples:** Fraud detection, recommendations

### 📱 Edge Deployment

**When:** Low latency, privacy concerns
**Challenges:**

- Limited resources
- Model optimization needed
- Model quantization

**Examples:** Mobile apps, IoT devices

### 🎯 Shadow Mode

**When:** Testing new model safely
**Process:**

- New model runs in parallel
- Predictions logged but not used
- Compare with production model

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

**Benefit:** No risk to users

# Model Monitoring

## 📊 Key Metrics to Monitor

**Performance Metrics:**

- Accuracy, Precision, Recall
- Latency (response time)
- Throughput (requests/second)
- Error rates

**Data Quality Metrics:**

- Feature distribution
- Missing values
- Data drift
- Outliers

## 🎯 Data Drift & Concept Drift

### 📉 Data Drift

**Definition:** Input data distribution changes
**Example:antml:parameter>**

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF